Auditoría del módulo de alimentación y nutrición

Fecha: **16 de octubre de 2026**. Base: árbol de trabajo existente, incluidos cambios locales; HEAD `e7bffc6`. Instrucciones: Agents.md.

Alcance: modelos, servicios, controladores, rutas, permisos, planes comerciales, interfaz, integración con lotes/forrajes/finanzas y PWA. Se auditó código ejecutable; la documentación se utilizó como contraste, no como prueba de funcionalidad. No se modificó lógica de negocio ni se accedió a datos productivos.

1. Resumen Ejecutivo

El módulo es útil como registro operativo de alimentación por lote, pero necesita correcciones antes de considerarlo confiable para trazabilidad histórica. Tiene catálogo, presentaciones, raciones, asignaciones, suministros, sobrantes y fotografías de datos al registrar. No es un formulador nutricional ni un inventario de bodega y tampoco calcula costos alimentarios.

Se identificaron **14 hallazgos: 6 P1 y 8 P2**. P1 significa prioridad alta por aislamiento, escrituras parciales, historia alterada o control comercial; P2 corresponde a inconsistencias funcionales, trazabilidad y limitaciones operativas. No son puntuaciones CVSS.

Los riesgos principales son la creación de documentos en una organización/finca indicada por el cliente, falta de atomicidad e idempotencia, uso de población actual para entregas históricas, correcciones que cambian conversiones antiguas y ausencia de validación de suscripción vigente en las rutas propias del módulo.

Capacidades y controles confirmados

- Alimentos, planes, raciones y suministros están modelados **por organización y finca**, no como catálogo compartido entre todas las fincas.
- Las consultas ordinarias incorporan contexto de organización/finca. Las asignaciones consultan lote y plan/ración bajo ese mismo contexto y validan especie, propósito y estado activo.
- No existen rutas DELETE de alimentos, raciones, planes o suministros. Se desactivan catálogos; no se encontró un flujo de borrado físico que deje huérfanos históricos.
- Las presentaciones omitidas al editar un alimento se conservan inactivas. Los suministros guardan nombres, factor usado, plan, ración y número de animales como snapshots.
- KG y toneladas se convierten directamente. Otras unidades requieren presentación o factor; no se inventa un factor ausente.
- Solo se calcula consumo estimado cuando **todos** los componentes tienen sobrante; en caso contrario se conserva `null`.
- Las correcciones exigen motivo y guardan valores anteriores/nuevos. `SuministroAlimentacion` activa `optimisticConcurrency`; esto protege la edición concurrente de ese documento, no vuelve transaccional el flujo completo.
- Existen índices únicos parciales para una asignación activa de plan y una de ración por lote. Se verificó la declaración en el esquema, no su instalación en una base desplegada.
- Trabajador puede registrar suministros en línea; no gestionar catálogos ni corregir entregas. Veterinario tiene lectura en el módulo ordinario.

Capacidades que no existen en este flujo

No se encontró balance de dietas, porcentajes de ingredientes que sumen 100%, requerimientos nutricionales, proteína calculada, consumo de materia seca de la ración, costo por kg/día/cabeza, prorrateo contable, descuento de inventario, conversión alimenticia basada en suministros, programación diaria automática ni cola offline de alimentación. No se

califican todas estas ausencias como defectos: la documentación del módulo también declara un alcance operativo básico.

Matiz sobre materia seca: Potreros sí calcula materia seca de un **corte** a partir del porcentaje ingresado. Alimentación no utiliza ese dato para convertir o balancear el suministro. **Matiz sobre finanzas:** Finanzas registra compras y precios de productos por su propio flujo; no hay enlace ejecutable desde un suministro hacia `MovimientoFinanciero`.

Verificación

- `npm.cmd test`, ejecutado en `backend`: **231 pruebas aprobadas, 0 fallidas, 0 omitidas**, salida 0.
- Catorce comprobaciones adicionales aisladas, detalladas en la sección 6, utilizando funciones reales y almacenamiento sustituido por dobles en memoria. Para altas y filtros se ejecutaron validación y hooks reales de Mongoose; para frontend se ejecutó la función real de inicialización extraída del archivo.
- No se conectó a MongoDB ni se probó concurrencia contra un servidor real. No se ejecutaron peticiones HTTP autenticadas ni navegación en navegador. La comprobación de suscripción recorrió los handlers de la ruta con un contexto simulado; la autenticación se inspeccionó separadamente. La suite verde no cubre los hallazgos completos.

2. Catálogo Exhaustivo de Funcionalidades Activas

La madurez describe el alcance observado. «Básico» significa que existe y tiene utilidad acotada; «Incompleto» indica un flujo con problemas o una integración faltante. No se asigna «100% Funcional» a flujos sin validación integral contra base de datos y navegador.

Submódulo / Función	**Descripción en Código**	**Estado Backend**	**Estado Frontend**	**Nivel de Madurez (100% Funcional / Básico / Incompleto)**
Catálogo de alimentos	Nombre normalizado único por finca; siete tipos; estado y descripción	Crear/listar/editar; inicializa 13 alimentos cuando no hay ninguno	Catálogo dentro de Raciones; alta y edición	Básico
Presentaciones	SACO, PACA, BALDE, CARRETA, OTRA; kg por unidad, estimada y activa	Conserva presentaciones omitidas como inactivas	Editor de presentaciones y factores	Básico
Metadatos nutricionales	`metadataNutricional` libre y `catalogoPasto` opcional	Persistencia sin contrato nutricional ni cálculo	Sin editor específico de nutrientes o vínculo al catálogo de pastos	Incompleto
Raciones	Ingredientes con cantidad/unidad/base; especie, propósito, etapa y plan opcional	Crear/listar/obtener/editar; exige componentes y alimentos activos	Editor y vista de ingredientes	Básico
Planes	Ficha de estrategia con una etapa y manejo; fechas de vigencia opcionales	CRUD sin DELETE; asignación a varios lotes	Alta/edición básica; cambio de plan desde Lotes	Básico

Submódulo / Función	Descripción en Código	Estado Backend	Estado Frontend	Nivel de Madurez (100% Funcional / Básico / Incompleto)
Asignación e historial de plan	Cierra asignación anterior, crea nueva y evento	Compatible por especie/propósito; índice de relación activa	Plan actual e historial en Lotes; sin interfaz masiva equivalente al endpoint	Básico
Asignación e historial de ración	Relación independiente del plan del lote	Actual e historial consultables; evento de cambio	Asignar desde detalle de ración; lote muestra ración actual	Básico
Precarga de suministro	Multiplica cantidades por cabeza por número actual de miembros al seleccionar lote	Backend recibe cantidades reales; no calcula la entrega planificada	Precarga editable; admite entrega sin ración	Básico
Registro real	Lote activo con miembros; fecha, ingredientes, responsable y snapshots	Guarda entrega y después EventoLote	Botón Registrar alimentación / Registrar suministro	Básico
Conversión física	KG directo, tonelada ×1000; presentación/factor para otras unidades	Funciones ejecutables; debilidades de coherencia de unidad	Selector de presentación o factor manual	Básico
Sobrantes y consumo	Entregado menos sobrante, solo con datos completos	Totales y kg por cabeza por registro	Captura sobrante, muestra algunos detalles; no panel completo de consumo	Básico
Corrección	Motivo obligatorio, historial anterior/nuevo, control optimista	Existe; recalcula con catálogo actual y no actualiza evento	Edición habilitada a administradores/encargados; problemas de contexto	Incompleto
Historial de suministros	Listado general y por lote, ordenado por fecha	Límites 500/200 sin paginación navegable	Tabla, filtro local por lote y modal; no visor del historial de correcciones	Incompleto
Resumen del día	Número de entregas, lotes distintos y último registro	Usa día del servidor y finca activa	Tarjeta del Dashboard; Dashboard reservado a Administrador	Básico
Origen de forraje	Referencia a un CorteForraje existente	Lista últimos 100; valida existencia del ID	Selector de corte en cada ingrediente suministrado	Básico
Bitácora	Evento de asignación y evento de entrega	EventoLote; sin EventoAnimal por cada cabeza	Cronología de lote; evento no se reconcilia al corregir	Incompleto

Submódulo / Función	Descripción en Código	Estado Backend	Estado Frontend	Nivel de Madurez (100% Funcional / Básico / Incompleto)
Protocolo de engorde	Paso `CAMBIO_RACION` llama al mismo servicio de asignación	Integración PRO con permiso/asignación de actividad	Selector en ejecución de protocolo	Básico
Gestión de existencias y compras	No hay stock ni vínculo Alimento–producto comprado	No implementado en este módulo	No disponible	Incompleto
Costos y contabilidad del suministro	No hay precio ni costo calculado en las entidades alimentarias	No crea MovimientoFinanciero	No disponible	Incompleto
Nutrición y conversión alimenticia	Sin cálculo de nutrientes ni relación alimento/ganancia	IEE existente usa otros componentes	No hay formulador ni reporte alimentario avanzado	Incompleto
Captura offline	No existe store ni operación de sincronización de suministros	No hay protocolo de reintentos idempotentes	Se deshabilita alta sin conexión	Incompleto

Artefactos y campos efectivos

Artefacto	Campos / responsabilidad
Alimento.js	`nombre`, `nombreNormalizado`, `tipo` (FORRAJE/CONCENTRADO/GRANO/SUBPRODUCTO/MINERAL/SUPLEMENTO/OTRO), `catalogoPasto`, `descripcion`, `activo`, `presentaciones`, `metadataNutricional`, `creadoPor`, tenant y timestamps. No tiene campos tipados de precio, proteína o porcentaje de materia seca.
Racion.js	Nombre, especie, propósito, etapa, `planAlimentacion` opcional, componentes, estado, observaciones. Cada componente: alimento, cantidad, unidad, presentación, base y observaciones. No porcentajes.
PlanAlimentacion.js	Nombre, especie, propósito, **una** etapa, manejo PASTOREO/CORTE_ACARREO/ESTABULADO/MIXTO, vigenciaInicio/Fin, estado y observaciones. No secuencia de etapas ni duración de cada etapa.
Lote.js	No contiene plan/ración/historial embebidos. Las respuestas se enriquecen consultando las colecciones de relaciones y suministros.
AsignacionPlanAlimentacion, AsignacionRacionLote	IDs de lote y plan/ración, inicio/fin, activo, usuario y observaciones. Una relación activa de cada clase por lote.

Artefacto	Campos / responsabilidad
SuministroAlimentacion.js	Lote, fechaHora, plan/ración y snapshots, etapa, cantidadAnimalesSnapshot, detalles, totales, responsable, registrador, corrector, motivo e historial. Por ingrediente: alimento/nombre, cantidad/unidad de entrada, presentación/nombre/factor, kg, tipoMedicion, sobrante, origen y observaciones.
CorteForraje.js	Potrero, forraje, fecha, kg verdes, porcentaje y kg de materia seca, destino y responsable. Es el origen opcional de un suministro; no es una entrada de bodega.

Backend principal: alimentacion-service, planAlimentacion-service, alimentacion-controller, planAlimentacion-controller, lote-service.

Frontend principal: Alimentacion.js, FormularioAlimentacion.js, Lotes.js, ListaUsuario.js, api.js, ProtocolosEngorde.js.

APIs encontradas

Montaje autenticado en app.js; permisos por endpoint en alimentacionRoutes.js y loteRoutes.js.

Ruta	Métodos	Función
`/api/alimentacion/alimentos`	GET, POST	Catálogo y alta; GET puede inicializarlo
`/api/alimentacion/alimentos/:id`	PUT	Editar/desactivar
`/api/alimentacion/raciones`	GET, POST	Listar/crear
`/api/alimentacion/raciones/:id`	GET, PUT	Detalle/editar/desactivar
`/api/alimentacion/planes`	GET, POST	Listar/crear; alta admite `lotes` opcional
`/api/alimentacion/planes/:id`	GET, PUT	Detalle con asignaciones/editar
`/api/alimentacion/planes/:id/asignar-lotes`	POST	Asignación de un plan a varios lotes
`/api/lotes/:id/plan-alimentacion`	GET, POST	Plan actual y cambio
`/api/lotes/:id/historial-alimentacion`	GET	**Historial de planes**, no de entregas
`/api/lotes/:id/racion`	GET, POST	Ración actual y cambio
`/api/lotes/:id/historial-raciones`	GET	Relaciones anteriores de ración
`/api/lotes/:id/suministros`	GET	Últimas 200 entregas del lote; no hay POST aquí
`/api/alimentacion/suministros`	GET, POST	Listado general filtrable y registro real
`/api/alimentacion/suministros/:id`	GET, PUT	Detalle y corrección con motivo
`/api/alimentacion/resumen-hoy`	GET	Cantidad de suministros, lotes distintos y último
`/api/alimentacion/origenes/cortes`	GET	Últimos 100 cortes de la finca

3. Flujo Operativo Real (Alimento → Ración → Plan → Suministro)

Catálogo y formulación operativa

1. Al abrir Alimentación se consultan planes, raciones, alimentos, lotes activos y suministros en paralelo. Si no existe ningún alimento en la finca, el GET inicializa 13 entradas.
2. Se crea el alimento y, si corresponde, presentaciones como «saco de 40 kg». La unidad no es una propiedad única del alimento: se elige por ingrediente/entrega y se convierte a kg.

3. Se crea una ración con uno o más alimentos. Bases: `POR_ANIMAL_DIA`, `POR_LOTE_DIA`, `LIBRE_ACCESO`, `OTRO`. Se aceptan cero y cantidades fraccionarias; para libre acceso la UI envía cero. No hay mezcla porcentual ni suma a 100% que validar. Se comprueba existencia/actividad del alimento, pero no toda la coherencia de presentación y plan asociado.
4. El plan es una ficha independiente de estrategia. Una ración puede apuntar opcionalmente a él. **Asignar un plan a un lote no selecciona automáticamente una ración**; ambas relaciones se gestionan por separado. Tampoco se exige que la ración pertenezca al plan actual.
5. La asignación cierra la relación anterior y crea una nueva y su EventoLote. Si ya está asignado el mismo ID, devuelve la relación existente. Las consultas de ambas entidades se filtran por finca y organización activas; no basta con conocer un ID de otra finca para asignarlo por el camino normal.

Registro de entrega y cambios poblacionales

6. El usuario selecciona lote en Registrar alimentación. La UI obtiene la ración actual y precarga cantidades. Para una receta de **5 kg/cabeza**, 20 miembros precargan **100 kg**; si se vuelve a seleccionar con un listado actualizado de 30 miembros, precarga **150 kg**. Con 25, precarga **125 kg**. Las ventas confirmadas y bajas cierran pertenencias, por lo que un nuevo conteo refleja la salida: ventaAnimalController, animal-controller.
7. Esa precarga no es una programación automática: el usuario puede modificarla y el backend no impone la ración. Un formulario ya abierto no vuelve a multiplicar si cambia la población; en cambio el backend vuelve a contar al guardar. Así, una precarga antigua de 100 kg guardada cuando hay 30 miembros registra 3.333 kg/cabeza, no una entrega automática de 150 kg.
8. El servidor exige lote activo y al menos una pertenencia activa. Cuenta **registros individuales de PertenenciaLote**; no incorpora remanentes colectivos de Camada. No debe presentarse este registro como alimentación completa de lechones todavía no individualizados.
9. Convierte detalles, valida sobrantes, obtiene plan/ración activos y guarda snapshots. Se permite suministro sin plan ni ración y varios suministros legítimos en un día. Si el cliente envía ración, comprueba que sea la activa; no exige que alimentos/cantidades reales coincidan con la receta.
10. Guarda un EventoLote de entrega. No crea un EventoAnimal por cada integrante ni conserva una lista de animales alimentados dentro del suministro: la atribución individual depende del historial de pertenencias.

Fórmulas realmente ejecutadas

En convertirAKg / construirTotales:

```
kg(KG) = cantidad
kg(TONELADA) = cantidad × 1000
kg(otras unidades) = cantidad × factor de presentación o factor manual
totalKgSuministrados = suma de kg de los componentes
kgSuministradosPorCabeza = totalKgSuministrados / cantidadAnimalesSnapshot
consumoEstimado = suma(kg − sobranteKg), solo si todos tienen sobrante
consumoEstimadoPorCabeza = consumoEstimado / cantidadAnimalesSnapshot
```

Los totales se redondean a tres decimales. No se multiplica por porcentaje de materia seca, precio ni requerimiento nutricional. «Por cabeza» es promedio del grupo **por entrega**, no consumo medido de cada animal ni total diario consolidado.

Forraje, costos, historial y resumen

- En registrarCorte, Potreros obtiene el forraje según cobertura y fecha; calcula `kgMateriaSeca = kgVerdes × porcentaje / 100` si se indicó porcentaje. Alimentación solo consulta estos cortes y permite referenciar uno

desde el detalle real. El esquema Racion no referencia cortes: estos se seleccionan al suministrar, no como lote de stock de la receta.

- La ruta llamada `cortesDisponibles` no calcula disponibilidad: devuelve los últimos 100 cortes, aunque ya se hayan referenciado. No valida saldo, correspondencia con ingrediente ni fecha del suministro. El origen COMPRA está admitido en el esquema, pero no tiene selector de compra en UI ni comprobación de documento financiero.
- Alimentos/raciones/suministros no tienen precio tipado ni cálculo de costo. MovimientoFinanciero dispone de productos, cantidades, precios y categorías, pero el servicio alimentario no lo invoca. No existe un «costo calculado aislado»: **el costo del suministro no se calcula**. Una compra en Finanzas y una entrega en Alimentación son registros independientes.
- No hay DELETE de los cuatro recursos principales. Desactivar conserva referencias y snapshots; no se observó el supuesto borrado destructivo de alimentos/raciones suministradas. El riesgo real está en edición/desactivación y consistencia del historial, no en un endpoint DELETE inexistente.
- `resumen-hoy` cuenta entregas y lotes distintos y devuelve el último registro, dentro del día del servidor. No informa kg totales del día, comida pendiente, cumplimiento de receta ni lotes sin alimentar. Un registro no demuestra que se completó la alimentación de todo el día.

4. Evaluación de Riesgos e Inconsistencias (P1 y P2)

ALI-01 — P1: altas con organización/finca controlables por el cliente

Evidencia: crearAlimento / crearRacion, crear plan expanden el cuerpo completo. El plugin solo completa tenant si está ausente; no reemplaza uno explícito ni verifica pertenencia al guardar.

Efecto: un Administrador/Encargado puede crear alimento, plan o ración con `organizacionId`/`fincaId` ajenos. En la ración se pueden validar ingredientes bajo la finca actual y guardar el documento bajo otra, generando referencias cruzadas. R1 reprodujo las tres altas hasta `collection.insertOne`, con hooks y validación reales y almacenamiento simulado. Las consultas sí quedan aisladas: no se afirma un bypass general de lectura. `creadoPor` sí se fija al usuario autenticado.

Corrección: DTO explícito; tenant siempre derivado del contexto e inmutable; validar referencias en el mismo ámbito. Probar las tres altas con tenant manipulado y confirmar ausencia de inserción ajena.

ALI-02 — P1: suministro y bitácora sin atomicidad ni idempotencia

Evidencia: crearSuministro inserta suministro y luego EventoLote sin transacción, compensación ni clave de operación.

Efecto: un fallo del evento devuelve error con entrega ya persistida; reintentar duplica kilos. R5 dejó dos registros tras fallo y reintento, con un solo evento. También se admite repetir una misma petición exitosa. Esto es distinto de los múltiples suministros legítimos diarios: no corresponde imponer unicidad lote/día.

Corrección: persistencia atómica de entrega/evento e idempotencia por operación; devolver el resultado existente al reintentar. Probar fallo entre escrituras y pérdida de respuesta posterior al commit.

ALI-03 — P1: asignaciones parciales y relaciones cerradas ante fallos

Evidencia: asignarRacionLote, asignarPlanALotes. Guardan cierre anterior antes de crear nueva relación/evento; múltiples lotes se recorren secuencialmente. El alta de plan también puede guardar plan antes de fallar su asignación.

Efecto: si falla la nueva relación, queda cerrado el vínculo anterior; si falla el evento, cambia la relación aunque el cliente reciba error. Una asignación masiva puede aplicar a algunos lotes y fallar después. El índice único evita dos relaciones activas si está instalado, pero no revierte cierres ni eventos. R8 reprodujo el cierre persistido seguido de error.

El paso CAMBIO_RACION hereda el riesgo: ejecuta el cambio antes de guardar tarea/ciclo.

Corrección: definir atomicidad del conjunto solicitado y envolver cierre/alta/evento; mapear conflictos de índice; coordinar el resultado con el protocolo. No devolver éxito parcial como error indistinguible.

ALI-04 — P1: fecha histórica con población y asignaciones actuales

Evidencia: crearSuministro usa `activo:true` para miembros, plan y ración, sin relacionarlos con `fechaHora`. actualizarSuministro permite cambiar fecha manteniendo población y contexto del registro. Las asignaciones y vigencias no validan intervalos.

Efecto: entregar 100 kg cuando había 20 animales, pero registrarlo después de que el lote creció a 30, produce 3.333 en lugar de 5 kg/cabeza para esa entrega. Se aceptan entregas anteriores a la apertura del lote y futuras; no se captura retrospectivamente en un lote ya cerrado. Un plan/ración con inicio futuro pasa a ser «actual» inmediatamente por el flag activo; una fecha de cambio anterior al inicio cierra el intervalo al revés. La vigencia del plan no se usa para seleccionar la asignación.

R6 comprobó fecha anterior al lote y futura con conteo actual; R2 aceptó vigencia invertida; R8 aceptó fin anterior al inicio. Una carrera entre conteo, cambio de miembros y guardado también puede producir snapshots de momentos distintos; no se probó contra MongoDB.

Corrección: resolver pertenencia, etapa y asignaciones a fecha de entrega; separar fecha operativa de fecha de captura; definir política para históricos/cerrados y futuro. Corregir fecha debe validar o reconstruir coherentemente el contexto. Asegurar una lectura consistente durante cambios de miembros.

ALI-05 — P1: correcciones reinterpretan conversiones y dejan eventos antiguos

Evidencia: actualizarSuministro vuelve a preparar todos los detalles recibidos. prepararDetallesSuministro toma la presentación **actual**; su factor prevalece sobre el snapshot que envía la UI. No hay actualización del EventoLote.

Efecto reproducido R7: 2 sacos de 40 kg registrados como 80 kg; se modifica el catálogo a 50 kg por saco; al corregir observaciones con los detalles reenviados, quedan 100 kg. La bitácora conserva 80 kg. Si cambia fecha, también queda la fecha anterior en el evento. Se conserva el historial anterior en el suministro, pero no evita reinterpretar involuntariamente la entrega.

Si la presentación histórica se desactiva, el editor vuelve a enviarla y el servicio la rechaza. Se admite alimento histórico inactivo en la corrección, pero no presentación histórica inactiva, y la UI solo lista alimentos activos.

Corrección: conservar factor/nombre históricos de líneas no modificadas; diferenciar modificación de cantidad, presentación y observación; permitir contexto histórico válido. Sincronizar evento o registrar corrección explícita enlazada; mostrar historial de cambios al usuario.

ALI-06 — P1: las rutas alimentarias no exigen suscripción vigente

Evidencia: auth calcula `req.planVigente` pero continúa para una organización activa con plan suspendido/cancelado. alimentacionRoutes solo aplica permisos de rol; no usa `requireFeature('planesAlimentacion')` ni `asegurarPlanVigente`. Los servicios tampoco lo comprueban.

Efecto: las operaciones directas alimentarias siguen autorizadas por rol aun con `planVigente=false`. R10 ejecutó permiso y controlador reales de POST alimentos en ese estado y obtuvo 201. No fue una prueba HTTP completa. El listado general de lotes tiene otras consultas de plan que pueden bloquear la UI, pero eso no protege la API alimentaria directa.

Corrección: aplicar vigencia y capacidad en servidor, incluidas asignaciones desde rutas de lotes. Definir expresamente si la lectura histórica se permite tras suspensión. No se exige subir de plan para captura básica: la capacidad está habilitada en los cuatro planes vigentes.

ALI-07 — P2: contrato insuficiente de cantidades, presentaciones y medición

Evidencia: validarDetallesRacion, convertirAKg, prepararDetallesSuministro.

- La ración acepta cantidad 0 o null convertible a cero; no valida que presentación exista, esté activa o coincida con unidad. Puede guardarse una receta que no tiene conversión ejecutable al suministrar.
- En suministro se valida existencia/actividad de presentación, pero no `presentacion.unidad === unidadIngresada`. R3 usó SACO de 40 kg para 2 PACAS y calculó 80 kg.
- La API acepta suministro con cantidad cero aunque el formulario exige al menos 0.001. Un registro de cero puede aumentar el contador diario. No se determina aquí que deba prohibirse todo cero en una receta de libre acceso; la política necesita distinguir receta y entrega real.
- `tipoMedicion` puede decir PESADA aunque los kilos provengan de una presentación/factor; solo `presentacion.estimada` fuerza ESTIMADA. Un factor manual no queda automáticamente clasificado como estimación.

Corrección: validar unidad/presentación/base en el servidor; decidir límites de precisión y totales positivos para entrega; derivar o comprobar método de medición. La UI debe advertir antes de guardar recetas sin conversión.

ALI-08 — P2: relaciones y desactivaciones no conservan coherencia operacional

Evidencia: crear/actualizarRacion no valida existencia, finca, actividad ni compatibilidad del plan opcional; las relaciones de plan/ración del lote son independientes. cerrarLote cierra AsignacionPlanAlimentacion, pero no AsignacionRacionLote. actualizar lote revisa miembros/plan, no ración activa al cambiar clasificación.

Efecto: una ración puede referenciar plan inexistente o incompatible; un lote cerrado conserva «ración activa» y aumenta `lotesActivos` del catálogo. Una ración desactivada continúa como asignación activa y se toma en nuevos suministros sin volver a validar su actividad. Un lote vacío sin plan puede cambiar clasificación conservando ración incompatible.

Desactivar un plan con relaciones exige `confirmarDesactivacion` en API, pero el formulario no ofrece ese paso: queda en error 409. La API, con confirmación, conserva las asignaciones activas; debe aclararse si «inactivo» impide solo nuevas asignaciones o también su uso futuro.

Corrección: validar referencias y cerrar ración junto al lote; definir semántica uniforme de desactivación; incorporar confirmación concreta en UI y advertencias de relaciones incompatibles. No convertir la etapa en restricción obligatoria sin una regla de negocio explícita.

ALI-09 — P2: origen de forraje y responsable con trazabilidad insuficiente

Evidencia: prepararDetallesSuministro solo verifica que el corte exista. crearSuministro acepta responsable arbitrario sin comprobar usuario/membresía/acceso a finca.

Efecto: se puede vincular un alimento MINERAL a un corte de forraje, un corte posterior a la entrega o un responsable inexistente/ajeno. R13 confirmó que el servicio acepta el origen sin contrastar tipo de alimento y que no consulta saldo. `registradoPor` sí es el actor autenticado; el defecto es la atribución adicional a responsable. No se demostró acceso general a datos de otra organización mediante esta referencia.

No descontar saldo es una capacidad ausente declarada, no una sobreventa de un inventario implementado. Sin embargo, llamar «disponibles» a todos los cortes puede inducir esa interpretación. La referencia COMPRA no se valida ni tiene una integración financiera ejecutable.

Corrección: validar responsable con acceso vigente a finca; comprobar coherencia de origen/tipo/fecha/destino o declarar la referencia solo informativa. Si se implementa stock, diseñar movimientos y reservas, no inferir saldo a partir del selector actual.

ALI-10 — P2: UTC, hora civil y día del resumen no son consistentes

Evidencia: hoyHora / inicializar usa ISO UTC recortado en `datetime-local`; envía texto sin offset. listado y resumenHoy construyen límites con zona del proceso, no con `Organizacion.zonaHoraria`.

Efecto: un instante de 06:00 en Costa Rica aparece como 12:00 en el editor (R11). El guardado de una hora escrita por el usuario depende de la zona del servidor; puede desplazar el instante. El resumen «hoy» puede cambiar de día a distinta hora que el productor. El filtro de fecha inicial parsea YYYY-MM-DD en UTC y el final añade fin de día local del servidor, creando límites inconsistentes.

Corrección: contrato de instantes con offset/UTC y conversión explícita en el editor; límites diarios en la zona de la organización. Probar medianoche y edición sin cambios, tanto con servidor UTC como con otra zona.

ALI-11 — P2: edición y errores de carga pueden cambiar el significado del formulario

Evidencia: FormularioAlimentacion, Alimentacion.cargar.

- Editar conserva `lote` como objeto poblado, aunque el select espera ID; no inicializa los nombres históricos de plan/ración. R11 confirmó la forma del estado, sin prueba de renderizado.
- El selector permite cambiar lote y precargar otra ración al editar, pero PUT no cambia lote/ración/snapshots: puede guardar nuevos ingredientes en la entrega del lote original. Debe bloquearse el cambio o implementar una reasignación explícita y validada.
- La inicialización elimina `observaciones` de cada ingrediente y la UI no las vuelve a enviar. Una corrección puede borrarlas.
- Un error al consultar ración se interpreta como «lote sin ración»; un error al consultar cortes se convierte en lista vacía. Se ocultan fallos reales de red/autorización.
- El cambio rápido de lote no cancela ni comprueba respuestas anteriores de ración; una respuesta tardía puede restaurar el lote/contexto previo.
- No hay vista previa de kilos convertidos, consumo o comparación planificado/real que detecte estos errores antes de guardar. Sí se muestran errores devueltos por el backend y se explica que faltan sobrantes para calcular consumo.

Corrección: estado de edición normalizado e histórico, lote fijo en corrección ordinaria, preservación de campos y mensajes diferenciados para vacío/error; descartar respuestas obsoletas. Verificar con pruebas de componentes y navegador.

ALI-12 — P2: listados truncados y filtrado por especie después del límite

Evidencia: listarSuministros / suministrosLote / cortesDisponibles limitan a 500/200/100 sin cursor, total ni aviso. Alimentacion solicita suministros sin especie y filtra por `lote.especie` después de recibirlos.

Efecto: 500 suministros recientes porcinos pueden ocultar todos los bovinos anteriores aunque se seleccione Bovino. No hay navegación hacia páginas antiguas; la paginación local de tabla no recupera esos registros. Cortes antiguos dejan de estar disponibles en el selector. El historial de especie depende además de la clasificación actual del lote, no de un snapshot de especie.

La apertura del módulo también solicita el listado completo de lotes enriquecido: lote-controller llama `obtenerDetalleLote` por lote, con múltiples consultas. Se observó crecimiento de consultas por lote; no se midió latencia ni carga máxima.

Corrección: filtros y paginación en servidor antes del límite; indicar total/continuación; endpoint ligero para seleccionar lote. Mantener el acceso por ID a referencias históricas fuera de la página actual.

ALI-13 — P2: captura de alimentación no disponible offline

Evidencia: offlineStorage no tiene store de alimentación; offlineSync solo procesa completar tarea y nota de bitácora. ListaUsuario deshabilita gestión/registro cuando no hay conexión. El service worker en vite.config.js precachea recursos estáticos; no implementa cola de suministros.

Efecto: el trabajador no puede capturar una entrega sin señal. Completar una tarea offline no crea SuministroAlimentacion. No es pérdida de una sincronización existente, sino una capacidad no implementada relevante para trabajo en corral.

Corrección: si la operación de campo lo requiere, incorporar borrador/cola con tenant, fecha real, operación idempotente, contexto de lote y conflictos; conservar estado local hasta confirmación. Resolver ALI-02 y ALI-04 antes de habilitar reintentos automáticos.

ALI-14 — P2: consultar el catálogo puede fallar por inicialización concurrente

Evidencia: inicializarCatalogo hace `exists → insertMany` de 13 alimentos; listarAlimentos lo invoca desde un GET permitido a roles de lectura.

Efecto: dos primeras consultas pueden observar catálogo vacío y competir por el índice único, haciendo fallar una petición de lectura. R14 reprodujo el flujo con barrera y error de índice simulado: una inicialización completó y otra propagó el error. Crear un solo alimento manual antes de inicializar también hace que nunca se agreguen los predeterminados faltantes. No se verificó la carrera contra MongoDB.

Corrección: inicialización explícita o idempotente por claves normalizadas; recuperar carreras conocidas y completar faltantes sin sobrescribir personalizaciones. Evitar que una lectura produzca un error por una siembra ya realizada en paralelo.

5. Alineación con el Modelo SaaS y los 4 Planes

Configuración comercial realmente encontrada

Los importes citados en las instrucciones no coinciden con los defaults del código. planes.js declara **6/12/20/35 USD**, configurables por variables de entorno; no se inspeccionó la configuración privada ni el precio efectivo de producción. Esta tabla describe el repositorio, no una tarifa externa verificada.

Capacidad / default mensual	Esencial — 6 USD	Gestión — 12 USD	Pro — 20 USD	Premium — 35 USD
Lotes, planesAlimentacion y bancosForrajeros	Sí	Sí	Sí	Sí
Analítica productiva	No	Sí	Sí	Sí
Analítica económica	No	No	Sí	Sí
Protocolos de engorde con CAMBIO_RACION	No	No	Sí	Sí
Reportes entre fincas	No	No	No	Sí
Captura offline de suministro / balance nutricional / costo alimentario	No implementado	No implementado	No implementado	No implementado

Alimentación básica no tiene un límite propio de raciones, alimentos o entregas. Los cuatro planes permiten la capacidad; los límites de animales/fincas/usuarios se administran por otras vías. Los catálogos alimentarios permanecen por finca incluso en Premium.

No hay comprobación de especie contratada al crear fichas de plan/ración; sí existe en el alta de lotes. Por ello puede catalogarse una ración de otra especie sin que eso autorice crear animales o lotes de esa especie. Conviene alinear la interfaz y el contrato, sin afirmar un bypass de cuota animal que no se observó aquí.

Roles efectivos

Fuentes: permisosRoles, rutas de alimentación/lotes y permisos frontend.

Rol	Ver alimentación y lotes	Crear/editar alimentos, raciones y planes	Asignar plan/ración ordinarios	Registrar entrega	Corregir entrega
Administrador	Sí	Sí	Sí	Sí	Sí
Encargado	Sí	Sí	Sí	Sí	Sí
Trabajador	Sí	No	No	**Sí**	No
Veterinario	Sí	No	No	No	No
Consulta	Sí	No	No	No	No
Contador	No	No	No	No	No

El trabajador puede usar la pestaña Suministros en Alimentación. El botón rápido «Registrar alimentación» del detalle Lotes depende de gestión general del lote y no se muestra al trabajador; es una incongruencia de acceso rápido, no una prohibición del flujo principal. No hay selector de responsable en el formulario ordinario: por defecto queda el usuario registrador.

Excepción de protocolo: con PRO/Premium, un trabajador o veterinario puede ejecutar un paso CAMBIO_RACION asignado a su usuario. rolPuedeEjecutarPaso y rutas del protocolo habilitan esa acción puntual; no se interpreta como permiso general de gestión alimentaria.

Valor real y posicionamiento

La captura básica aporta registro de qué se entregó, cuándo, a qué lote, con qué unidad y quién lo registró. Tiene sentido mantenerla en Esencial y permitir al trabajador ejecutarla, con correcciones supervisadas. Los snapshots y su auditoría son requisitos de integridad de esa captura, no deberían depender de una mejora comercial.

Los reportes existentes de lotes/engorde sí tienen barreras de analítica: reporte.js. Sin embargo, el IEE pondera cumplimiento de GMD, eficiencia de tiempo y supervivencia; **no mide conversión alimenticia**. La cadena `preparadoPara: ['conversionAlimenticia', 'eficienciaEconomica']` es metadato, no implementación. No hay una analítica de alimento/ganancia filtrándose a Esencial porque ese cálculo todavía no existe.

Como evolución propuesta, distinta de las capacidades actuales: Gestión podría incorporar consumo diario y comparación planificado/real; Pro, costos y eficiencia alimentaria con fuentes conciliadas; Premium, consolidación entre fincas. Antes se necesitan identidad de entrega, población histórica, unidades coherentes y trazabilidad contable. No debe registrarse automáticamente otro egreso al consumir alimento ya comprado: compra, existencia y consumo requieren políticas separadas para evitar duplicación contable.

6. Plan de Acción Priorizado

Remediaciones y criterios de cierre

Prioridad	Hallazgo	Tarea concreta	Verificación de aceptación
P1	ALI-01	DTO y tenant del contexto; comprobar referencias	Las tres altas rechazan/ignoran tenant ajeno y solo persisten en finca autorizada
P1	ALI-02	Transacción entrega/evento e idempotencia	Fallo entre escrituras no deja entrega huérfana; reintento de misma operación no duplica; otra entrega legítima del día sí se admite
P1	ALI-03	Cambio de asignaciones atómico y coordinación con protocolo	Fallo en alta/evento restaura relación anterior; multi-lote tiene resultado definido y recuperable
P1	ALI-04	Contexto a fecha y fechas consistentes	100 kg para 20 animales históricos siguen representando 5 kg/cabeza aunque hoy haya 30; futuro e intervalos invertidos se manejan según política explícita
P1	ALI-05	Corrección conserva conversiones históricas y reconcilia bitácora	Corregir una observación no convierte 80 kg en 100; motivo/historial/evento permanecen coherentes
P1	ALI-06	Vigencia/capacidad verificadas en todas las entradas	Plan suspendido no puede mutar alimentación por API directa; los cuatro planes vigentes conservan captura básica
P2	ALI-07	Validar unidad, presentación, base, cantidades y medición	PACA no utiliza presentación SACO; receta sin conversión se detecta; entrega cero tiene tratamiento explícito
P2	ALI-08	Cierre de relación de ración y validación de plan/contexto	Cerrar lote deja cero relaciones activas; plan ajeno/inexistente se rechaza; desactivación se confirma y explica
P2	ALI-09	Responsable autorizado y origen coherente	No se atribuye a usuario sin acceso; referencia de corte incompatible/futura se rechaza o se trata expresamente como informativa
P2	ALI-10	Fechas con zona y límites diarios por organización	Editar sin cambios preserva instante; resumen y filtros coinciden con el día de Costa Rica
P2	ALI-11	Corregir estado de formulario, errores y respuestas tardías	Edición mantiene lote y notas; error de ración no se muestra como ausencia; cambio rápido de lote conserva la última selección
P2	ALI-12	Filtros/paginación de servidor y listado ligero	Más de 500 entregas siguen navegables por especie; selector permite recuperar cortes antiguos por ID
P2	ALI-13	Definir e implementar captura offline si es requisito de campo	Entrega sin señal conserva fecha/contexto y se sincroniza una sola vez o presenta conflicto visible
P2	ALI-14	Siembra idempotente del catálogo	Dos primeras lecturas simultáneas no fallan y el catálogo no duplica ni omite defaults por una carrera

Antes de recalcular o migrar registros, identificar entregas duplicadas, conversiones históricas modificadas, vínculos de tenant incorrectos, asignaciones activas en lotes cerrados y discrepancias con EventoLote. Conservar el original y el motivo de cada reconciliación; no sobrescribir historia masivamente sin diagnóstico.

Comprobaciones aisladas ejecutadas

Los resultados siguientes son observaciones de código real con dobles de almacenamiento; no equivalen a una prueba end-to-end ni certifican índices desplegados. Se ejecutaron por scripts efímeros mediante `node -`, sin archivos de negocio modificados ni conexión a MongoDB.

Prueba	Resultado
R1 — Aislamiento de altas	Alimento, Racion y PlanAlimentacion llegan a insertOne con organización/finca ajenas suministradas en payload
R2 — Validación de receta/plan	Guarda receta con cantidad 0, presentación y plan no verificados; acepta vigencia de plan invertida
R3 — Unidad y medición	2 PACAS usan presentación SACO de 40 kg y conservan etiqueta PESADA: 80 kg
R4 — Aritmética válida	1.5 toneladas = 1500 kg; 150 kg / 30 cabezas = 5; falta de un sobrante mantiene consumo null
R5 — Fallo/reintento	Evento falla después de guardar; reintento deja dos suministros y un solo evento
R6 — Fecha/población/responsable	Fecha pre-lote y año 2999 pasan el servicio; usa 30 miembros actuales y responsable recibido sin validación de pertenencia
R7 — Corrección de conversión	Cambiar presentación de 40 a 50 kg y reenviar detalles convierte 80 kg en 100; evento conserva 80
R8 — Asignación fallida	Cierra la relación anterior antes de que falle la nueva; acepta fin anterior al inicio
R9 — Filtros de lectura	Hook real añade organización y finca activas a findById
R10 — Suscripción	Permiso y handler reales de POST alimentos aceptan planVigente=false y responden 201 en el contexto simulado
R11 — Inicialización frontend	Mantiene lote como objeto, pierde observación de ingrediente y presenta 12:00 civil para instante 06:00 de Costa Rica
R12 — Paginación	Controlador general aplica límite 500, sin cursor ni total de respuesta
R13 — Corte	Solo comprueba existencia por ID; acepta origen de corte en alimento MINERAL y repetir 10000 kg sin consultar fecha/saldo
R14 — Siembra concurrente	Con barrera y error de índice simulado, una de dos inicializaciones propaga error; la otra crea los 13 defaults

La suite existente incluye alimentacionOperativa.test.js y lotesAlimentacion.test.js. Cubre conversiones, sobrantes, compatibilidad, campos e índices declarados; deben añadirse pruebas de integración de permisos, tenant, fallos, fechas y concurrencia, además de pruebas de interfaz. Ejecutar nuevamente la suite completa después de las correcciones.

Estado: auditoría completada; remediaciones pendientes. Único entregable nuevo de esta fase: este informe.